

Case 1: The Last Astronaut

ALGORITHM TheLastAstronaut

INPUT N, K

IF $K < 2$ THEN

$K = 2$

ENDIF

// Inisialisasi daftar astronaut

FOR i FROM 1 TO N DO

 astronaut[i - 1] = i

ENDFOR

size = N

pos = 0

PRINT "Urutan eliminasi:"

WHILE size > 1 DO

 pos = (pos + K - 1) MOD size

 eliminated = astronaut[pos]

 PRINT eliminated

 // Hapus elemen dari array

 FOR i FROM pos TO size - 2 DO

 astronaut[i] = astronaut[i + 1]

 ENDFOR

 size = size - 1

 // Update nilai K

 IF eliminated MOD 2 == 0 THEN

$K = K + 2$

 ELSE

$K = K - 1$

 IF $K < 2$ THEN

$K = 2$

 ENDIF

 ENDIF

ENDWHILE

PRINT "Astronaut terakhir:", astronaut[0]

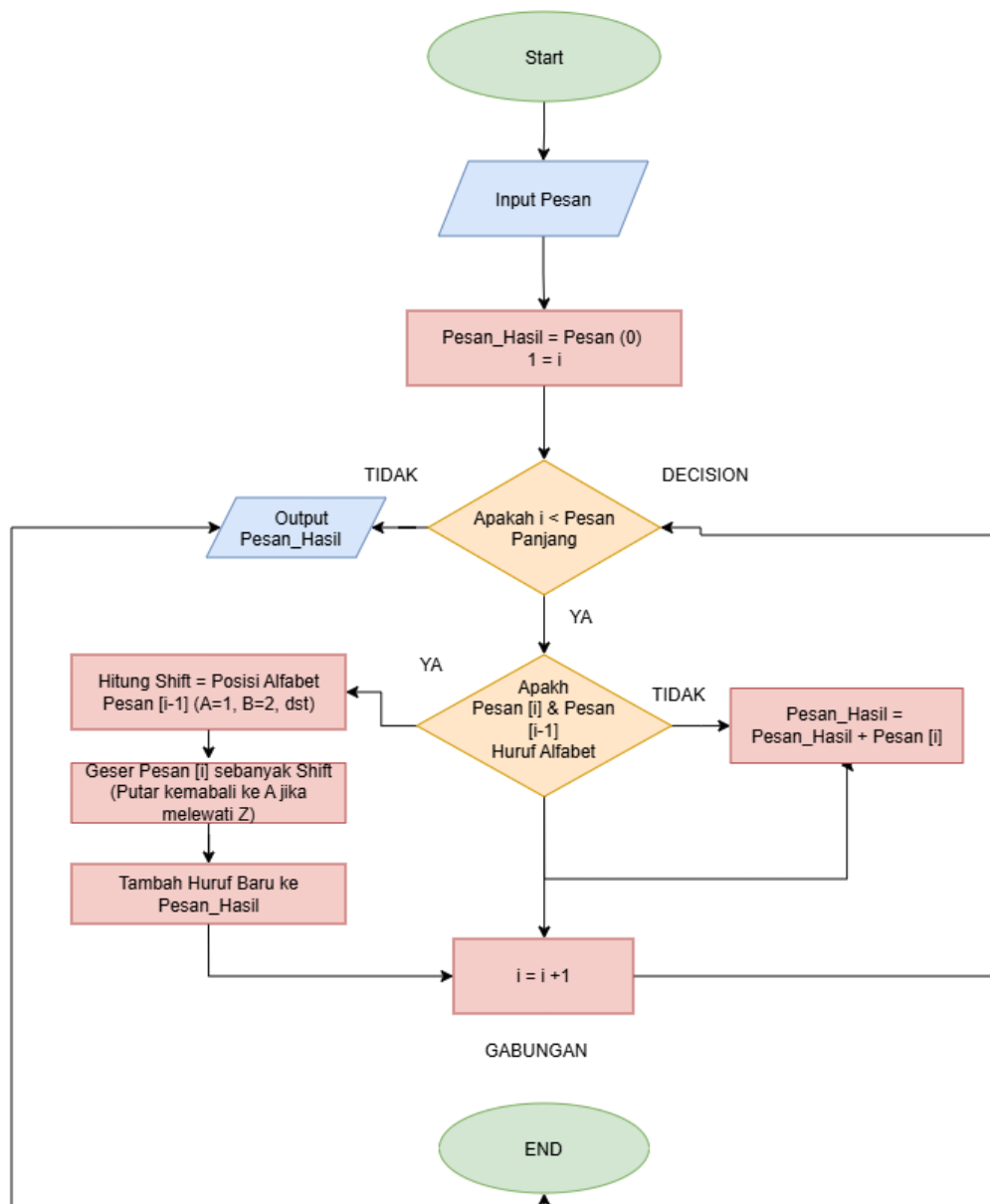
END

Penjelasan Logika Problem 1

Permasalahan ini diselesaikan menggunakan pendekatan simulasi lingkaran dinamis (Josephus Problem yang dimodifikasi). Seluruh astronot diberi nomor urut dari 1 hingga N dan dimasukkan ke dalam daftar/array. Perhitungan eliminasi dimulai dari posisi pertama dengan melangkah sebanyak $K-1$ langkah untuk menentukan astronot yang terhitung ke- K . Astronot yang berada pada posisi tersebut akan dicatat ke dalam urutan eliminasi, kemudian dikeluarkan dari daftar.

Setelah seorang astronot tereliminasi, nilai K akan diperbarui secara dinamis berdasarkan nilai astronot yang terekskusi. Jika nomor astronot tersebut genap, nilai K akan bertambah 2. Sebaliknya, jika nomornya ganjil, nilai K berkurang 1. Apabila penyesuaian tersebut menyebabkan K bernilai kurang dari 2, maka nilai K dipaksa menjadi 2. Proses perulangan dan penyesuaian nilai K ini terus berlanjut hingga hanya tersisa satu astronot terakhir di dalam daftar.

CASE 2 : ALIEN IN THE MIDDLE



Penjelasan Logika Problem 2

Algoritma penyandian ini bekerja dengan mentransformasi setiap karakter pada pesan berdasarkan nilai karakter yang berada tepat di sebelumnya. Karakter pertama dari pesan asli tidak mengalami perubahan sama sekali dan langsung dimasukkan ke dalam hasil sandi. Untuk karakter-karakter selanjutnya (mulai indeks ke-1), pergeseran dilakukan dengan menghitung posisi alfabet dari karakter saat ini ditambah dengan posisi alfabet dari karakter sebelumnya (A=1, B=2, ..., Z=26).

Untuk menangani pergeseran yang melebihi huruf 'Z', digunakan perhitungan aritmetika modular berukuran 26. Nilai gabungan dari karakter saat ini dan karakter sebelumnya dikurangi 1, dihitung sisa baginya terhadap 26, lalu ditambah 1 kembali agar

berada dalam rentang 1 hingga 26. Nilai numerik baru tersebut kemudian dikonversi kembali menjadi huruf kapital dan digabungkan ke dalam pesan hasil sandi hingga seluruh karakter selesai diproses.